

Demo videosu · çekim listesi

payper · Rise In × Stellar Pro Hackathon 2026 · Genesis Track

Amaç: jürinin "bunu teknik olarak gerçekten yaptınız mı" sorusunu videoyu izlerken cevaplamış olmak. Her adımda ne yapacağın, ekranda ne görünmesi gerektiği ve varsa o adımı kanıtlayan terminal komutu yazılı.

Çekim öncesi üç şey

1 · Tarayıcıda Freighter kurulu ve

testnet

ağında olsun, hesabında biraz XLM bulunsun.

2 · Terminali ayrı bir pencerede hazır tut, yazı boyutunu büyüt (kod okunabilsin).

3 · `cd ~/Downloads/payper` ile proje dizinine geç — komutlar oradan çalışıyor.

Bölüm 1 · Ürün çalışıyor

01 Ana sayfa

Yap: `payper.live` adresini aç, yavaşça aşağı kaydır.

Ekranda: Canlı site, "STELLAR TESTNET · LIVE" etiketi, alt bantta finanse edilen tutar ve fatura sayısı.

Söyle: "Tedarikçi malı sevk etmiş, maaşı ödemiş, parayı doksan gün bekliyor. O açığı kapatacak sermaye yurt içi bankaların bilançosunda. Biz o kapıyı USDC tutan herkese açıyoruz."

02 Cüzdan ile giriş — Freighter açılır

Yap: Sağ üstten panele gir, **Connect wallet** → Freighter'ı seç.

Ekranda: **Freighter penceresi açılır** ve imza ister. Onayla. Panelde hesap adresin görünür.

Söyle: "Giriş SEP-10 challenge ve SEP-53 imzalı mesajla. Sunucu parolayı değil, imzayı görüyor."

KANIT · İMZA DOĞRULAMA SPEC VEKTÖRLERİYLE TEST EDİLİYOR

```
npm run test:sep53
```

Spesifikasyonun kendi üç test vektörünü bayt bayt üretiyor — kendi uydurduğumuz bir vektör kendi içinde tutarlı olur ama spec ile uyuşmazdı.

03 Fatura yükle — belge doğrulanır

Yap: **Upload invoice** ekranında örnek dosyalardan `GIB2026000000481.xml`'e tıkla.

Ekranda: Sağda belge açılır: tutar, vade, **ETTN**, doküman hash'i ve dört yeşil kontrol — UBL-TR yapısı, XAdES imza bloğu, SHA-256, ETTN tekiliği.

Söyle: "Belge imzalanmadan önce doğrulanıyor. ETTN, vergi idaresinin her faturaya verdiği tekil kimlik."

KANIT · DOĞRULAYICI BOZUK BELGELERİ REDDEDİYOR

```
npm run test:ubl
```

04 Zincire kaydet

Yap: **Register on chain** tuşuna bas, bekle.

Ekranda: İşlem hash'i ve **stellar.expert linki** belirir. Linke tıkla, işlemi explorer'da göster.

Söyle: "ETTN'in hash'i artık kontratta. Aynısı bir daha kaydedilemez."

05 Alıcı onayı — Freighter yine açılır

Yap: Hesap menüsünden **Buyer** rolüne geç, **Buyer approval** → **Acknowledge on chain**.

Ekranda: Cüzdan oturumundaysan **Freighter onay ister**. Onayla, yeşil teyit ve işlem linki gelir.

Söyle: "Faturada yazılı adres dışında kimse onaylayamıyor. Bu kontratın kuralı, arayüzün değil."

06 Fiyat — dört bileşen, ikisi canlı

Yap: **Quote** ekranına geç, bileşenlerin yanındaki etiketleri göster.

Ekranda: Dört satır. **Currency risk** ve **Treasury yield** yanında **live**, diğer ikisinde **configured** yazıyor.

Söyle: "Her bileşen kaynağını söylüyor. Parametreye düşen bir sayı canlı gibi gösterilemiyor — bunu iddia etmiyoruz, kontrat etiketliyor."

07 İskontoyu kilitle

Yap: Rolü **Supplier**'a al, **Accept and lock**'a bas.

Ekranda: Kilitlenen oran ve işlem linki. Tedarikçinin alacağı tutar liradan görünür.

Söyle: "Oran o anki kurdan kilitleniyor. Teklifin süresi var, dolarsa yeniden fiyatlanıyor."

08 Fonlama — asıl kanıt, Freighter imzalıyor

Yap: Rolü **Funder**'a al, **Funding board** → tutarı seç → **Fund this invoice**.

Ekranda: **Freighter açılır ve ne yetkilendirdiğini gösterir**. Onayla. Makbuzda taşınan USDC, imzalayan adres ve **signed in** Freighter satırı görünür.

Söyle: "Burada imzayı sunucu atmıyor. Sunucu çağırıyor kuruyor ve simüle ediyor, imzasız zarfı cüzdana veriyor. Onaylamazsam hiçbir şey zincire gitmiyor."

KANIT • SUNUCUNUN VERDİĞİ ZARFTA İMZA YOK

```
npm run test:wallet
```

- ✓ the envelope arrived unsigned – the server signed nothing
- ✓ signed with the wallet's own key
- ✓ accepted by the ledger
- ✓ the contract recorded 2.00000000 USDC from that signature

Test, sunucudan gelen zarfta **hiç imza olmadığını** doğruluyor. "Sunucu imzalamıyor" iddiası sözle değil testle bağlı.

Not: bu komut uygulamanın çalışıyor olmasını ister; canlıya karşı çalıştırmak için `TEST_BASE=https://payper.live npm run test:wallet`.

09 Anchor — para bankaya gidiyor

Yap: **Anchor** ekranı, **Supplier withdrawal** • **USDC** → **TRY** sekmesi, **Run the USDC** → **TRY flow**.

Ekranda: Altı adım tek tek yeşile döner: stellar.toml keşfi, cüzdan kimliği, kur kilidi, çekim talebi, zincir üstü ödeme, bankaya lira. Sağdaki terminal her isteği ve dönen statüyü yazar.

Söyle: "Anchor hakkında hiçbir şey kodumuzda yazılı değil. Alan adı yapılandırma, gerisi çalışma anında okunuyor. Başka bir anchor'a geçmek tek alan adı değişikliği."

KANIT • ANCHOR'IN KENDİ UÇLARI, BİZİM SUNUCUMUZ DEĞİL

```
curl -s https://tr-mock-anchor.fly.dev/.well-known/stellar.toml | head -20
```

```
curl -s https://tr-mock-anchor.fly.dev/sep6/info | python3 -m json.tool | head -25
```

Ekrandaki kutularla bu çıktıyı karşılaştır — aynı uçlar, aynı limitler.

10 Ürünün tek değışmezi — aynı fatura reddedilir

Yap: Upload invoice'a dön, örnek dosyalardan GIB2026000000481-KOPYA.xml'i seç.

Ekranda: Kırmızı **Refused** kutusu ve bu ETTN'i hangi faturanın tuttuğu. İmzalama aşamasına hiç gelinmiyor.

Söyle: "Bir ETTN, bir finansman. Bu bir veritabanı satırı değil, kontrat değışmezi. Lisanslı bir faktoring şirketi merkezi kayda bakabilir; Berlin'deki bir fonlayıcı bakamaz. Kontrat ona aynı garantiyi hiçbir yere üye olmasını istemeden veriyor."

Bölüm 2 · Terminal kanıtları

Bu bölümü ekranı terminale alıp çekebilirsin. Sırayla çalıştır, çıktıları göster. Hepsi doğrulandı, olduğu gibi kopyalanabilir.

11 DeFi yield gerçekten DeFindex mi

Söyle: "Kasa bizim yazdığımız bir kontrat değil, DeFindex'in kendi factory'sinden açılmış. Bunu iddia etmiyoruz, hash karşılaştırıyoruz."

ZİNCİRDEKİ KASANIN WASM HASH'I

```
curl -s https://api.stellar.expert/explorer/testnet/contract/CBXHELM65LG05400WBCIQKRVHJGQPSG6D2J5QSALXKYHJHR2J5RP0DCP | grep -o '"wasm":"[a-f0-9]*"'
```

DEFINDEX'İN KENDİ YAYIMLADIĞI HASH

```
curl -s https://raw.githubusercontent.com/paltalabs/defindex/main/public/testnet.contracts.json | grep -A3 hashes
```

İkisi de: f345228dca59c6605789620e9ec62ff4847a0927c33dac7581a955fe746016be

KASANIN KENDİ ADI – BİZİM HAZİNEMİZ İÇİN AÇILMIŞ

```
stellar contract invoke --id CBXHELM65LG05400WBCIQKRVHJGQPSG6D2J5QSALXKYHJHR2J5RP0DCP \
--source payper-admin --network testnet -- name
```

"DeFindex-Vault-payper Treasury"

12 Getiri gerçekten canlı mı okunuyor

Söyle: "Getiri bileşeni bir parametre değil. Kasada strateji olmadığı için bir Blend v2 havuzunun tedarikçilere ödediği oranı okuyoruz — USDC'nin Stellar'da borç vererek kazandığı şey."

HAZİNE ADAPTÖRÜNÜN OKUDUĞU ORAN

```
stellar contract invoke --id CAFZQFGPDEHV62AMPGNEMYHV6MEPAKFFPVCXDVAQA5Q4QWEFBIA3X4 \
--source payper-admin --network testnet -- blend_apy_bps
```

129

FATURA KONTRATININ GÖRDÜĞÜ HÂLİ, KAYNAĞIYLA

```
stellar contract invoke --id CB2EUFAFCDKWHCYBHGDTFNOHJEVYH3WKTCL40TGX5FUKP272GHQG3NBA \
--source payper-admin --network testnet -- treasury_apy_bps
```

[129,0] ← 0 = Source::Live, çağrı sırasında zincirden okundu

13 Deploy edilen kontrat bu kaynaktan mı derlendi

Söyle: "Zincirdeki wasm'ı yerelde derleyip yükledik. GitHub Actions temiz bir runner'da derledi, GitHub o hash'e imza attı, biz o artefaktı deploy ettik."

WASM'IN İÇİNDEKİ KAYNAK DEPOSU – ZİNCİRDEN OKUNUYOR

```
stellar contract info meta --network testnet \
--id CB2EUFAFCDKWHCYBHGDTFN0HJEVYH3WKTCL40TGX5FUKP272GHQG3NBA
```

```
source_repo: github:cansarihan/payper
home_domain: payper.live
```

CI'IN ÜRETTİĞİ ARTEFAKTIN HASH'İ İLE ZİNCİRDEKİ AYNİ Mİ

```
gh release download v1.0.2_contracts_invoice_payper-invoice_pkg0.1.0_cli27.0.0 -R cansarihan/payper
shasum -a 256 payper-invoice_v0.1.0.wasm
curl -s https://api.stellar.expert/explorer/testnet/contract/CB2EUFAFCDKWHCYBHGDTFN0HJEVYH3WKTCL40TGX5FUKP272GHQG3NBA | grep -o '"wasm":"[a-f0-9]*"'
```

İkisi de: 94dc49bef260c30989615cebf545596cb4370f02a4e19b3a8ea6350250c68af6

GITHUB'IN O HASH'E ATTIĞI İMZA

```
gh api repos/cansarihan/payper/attestations/sha256:94dc49bef260c30989615cebf545596cb4370f02a4e19b3a8ea6350250c68af6
```

14 Testler

Söyle: "Otuz altı kontrat testi. İçinde devredilebilir alacağın toplamı korumasından, fonlama defterinin yürünebilir kalmasına kadar."

```
npm run test:contracts
```

```
test result: ok. 36 passed; 0 failed
```

İSTERSEN HEPSİ

```
npm run test:ubl && npm run test:tranche && npm run test:sep53 \
&& npm run test:auth && npm run test:chirp
```

15 Uçtan uca akış, tek komut

Söyle: "Bu komut faturayı kaydediyor, alıcıya onaylatıyor, canlı fiyat okuyor, iki fonlayıcı anchor'dan USDC alıp fonluyor, vadede kapatıyor, oransal dağıtıyor ve sonunda aynı ETTN'ı tekrar kaydetmeyi deneyip reddediliyor. Altı adımın hepsi testnet'e gerçek işlem yazıyor."

```
npm run smoke
```

```
6. Aynı ETTN tekrar deneniyor
× REDDEDİLDİ · EttnAlreadyUsed
✓ Zincir akışı baştan sona çalıştı
```

Söylenmesi gerekenler

Bunları sen söyle, jüri bulmasın

Anchor bir

sandbox

— banka ve KYC bacağı simüle, Stellar bacağı gerçek testnet USDC.

Passkey

ile girildiğinde imza sunucuda kalıyor: WebAuthn secp256r1 üretiyor, Stellar Ed25519 bekliyor. Zincirde secp256r1 doğrulayan akıllı hesap gelene kadar passkey Soroban XDR imzalayamıyor.

Cüzdanla girildiğinde imza kullanıcının.

Kur riskini

henüz kimse taşıyor, fonlayıcının hakkının

hukuki kabı

yok, ve

uydurma bir fatura

yapısal kontrollerden geçer — kapı acknowledge(), ama muvazaayı durdurmuyor.

Çekim özeti

#	Adım	Kanıt
01	Ana sayfa	—
02	Cüzdan girişi · Freighter açılır	test:sep53
03	Fatura yükleme · belge doğrulanır	test:ubl
04	Zincire kayıt · işlem linki	stellar.expert
05	Alıcı onayı · Freighter açılır	—
06	Fiyat · iki bileşen Live	adım 12
07	İskonto kilitleme	—
08	Fonlama · Freighter imzalar	test:wallet
09	Anchor · altı SEP adımı	curl stellar.toml
10	Aynı fatura reddedilir	smoke
11	DeFindex hash eşleşmesi	iki curl
12	Getiri canlı · 129 bps	treasury_apy_bps
13	Yeniden üretilebilir derleme	gh + hash
14	36 kontrat testi	test:contracts
15	Uçtan uca akış	smoke

Sıra önemli: **önce fonlama (08), sonra anchor (09)**. Fonlama ödemeyi tedarikçiye geçiriyor, anchor çekimi de o bakiyeyi bankaya gönderiyor. Tersi olursa çekimde bakiye bulunmaz.